

Generative Forecasting with Joint Probability Models

A **Framework** that redefines how we predict the behavior of complex, chaotic dynamical systems.

**Probabilistic Machine Learning For Mechanics(Deep Generative Models)
Course Project**

Abhishek R Reddy
2025AMA2244

Generative Forecasting using **Joint** Probability Models

Contents

1. What is Chaotic Dynamical Systems?
2. Concepts related to Chaotic Dynamical System
3. Generative Forecasting with Conditional Probability
4. Generative Forecasting using Joint Probability Models
5. Uncertainty quantification from joint samples
6. Wasserstein drift (a UQ Metric)
7. Extension to Joint Generative Framework
 1. Conditional Joint Models.
 2. Latent optimal control
8. Evaluation of framework on Lorenz-63 model
 1. Generating the Data for Training
 2. NADE Implementation to Forecast Lorenz System
 3. Testing Methodology
 4. Distribution adherence test
 5. Estimation UQ metrics
 6. Implementation of Conditional Joint Model

Chaotic Dynamical System

- A chaotic dynamical system is a system that is **extremely sensitive to its initial conditions**.
- Because of this extreme sensitivity, even tiny changes at the start will accumulate over time and lead to completely different outcomes.
- Examples:
 - **Atmosphere**
 - Convection (Lorenz System)
 - Fluid Turbulence
 - Nonlinear Vibrations in Structures
 - Celestial Mechanics etc.,

Concepts related to Chaotic Dynamical System

Determinism (Non-Randomness):

- A chaotic system is **not** random.
- It is governed by strict physical laws. If we start with exactly the same initial conditions, you would get the same result.
- However, it is impossible to obtain the real-world values with infinite precision.

Non-Linearity

Chaos cannot exist in linear systems. Non-linear terms must exist in governing equations.

Phase Space

Phase space is a mathematical space where all possible states of a dynamical system are represented.

Attractor

- An attractor is a set of states in phase space toward which a dynamical system evolves over time.
- It often settles into a specific "shape" in phase space.
- In chaotic systems, attractors are strange attractors.

Example of a Strange Attractor



Lorenz attractor

Edward Lorenz developed the system as a simplified mathematical model for atmospheric convection.

The model describes how three key properties of this system change over time:

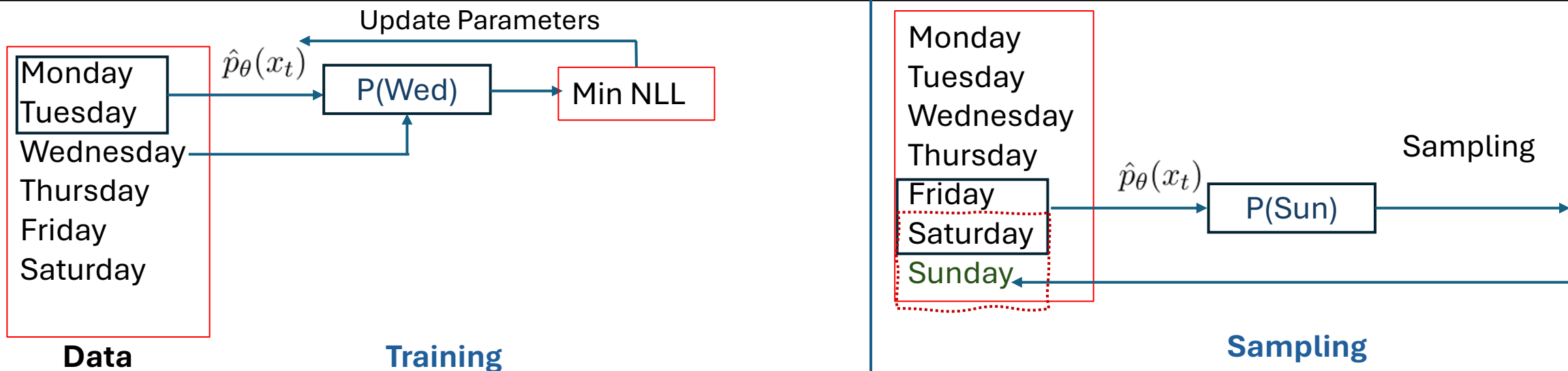
- x is proportional to the intensity of the convection (the rate of fluid flow).
- y is proportional to the temperature difference between the rising and falling air currents.
- z is proportional to the distortion of the vertical temperature profile from a linear one.

Generative Forecasting with **Conditional** Probability

- Forecasting using patterns in a **timed sequence of data** (past) to estimate the probability of **future events**. is
- Predicts the next state by looking only at the sequence of past observations.

$$\underbrace{\hat{p}_{\theta}(x_t)} \approx p(x_t \mid \underbrace{x_{t-\Delta t} = \tilde{x}_{t-\Delta t}, x_{t-2\Delta t} = \tilde{x}_{t-2\Delta t}, \dots}_{\text{Timed Sequence of Data}}),$$

The model approximates the Conditional PDF, using historical observations as constraints to sample the next state.



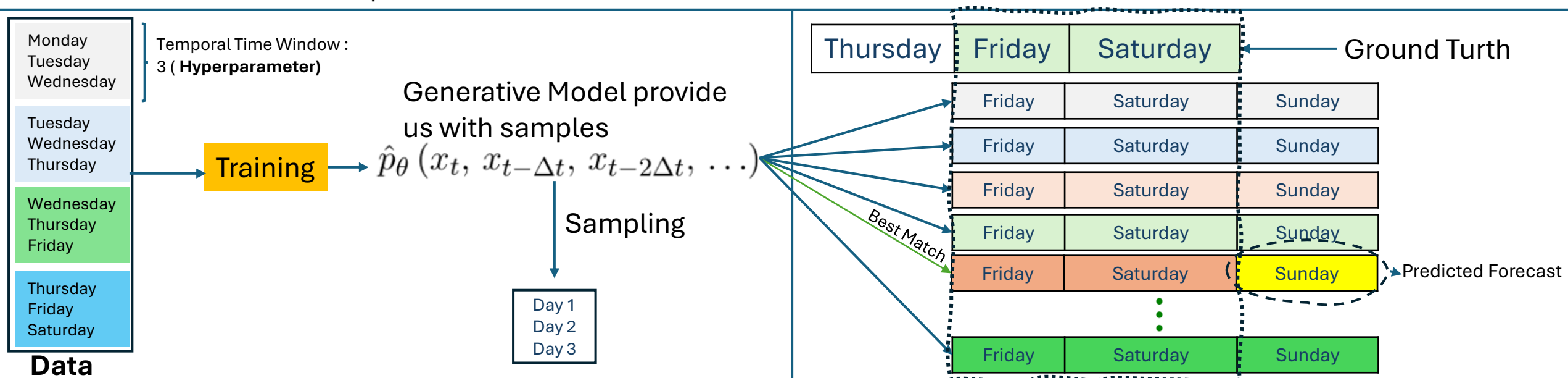
Generative Forecasting using **Joint** Probability Models

$p(x_t \mid x_{t-\Delta t} = \tilde{x}_{t-\Delta t}, x_{t-2\Delta t} = \tilde{x}_{t-2\Delta t}, \dots)$ can be written using Product Rule of Probability as:

$$p(x_t \mid x_{t-\Delta t} = \tilde{x}_{t-\Delta t}, x_{t-2\Delta t} = \tilde{x}_{t-2\Delta t}, \dots) = \frac{p(x_t, x_{t-\Delta t} = \tilde{x}_{t-\Delta t}, x_{t-2\Delta t} = \tilde{x}_{t-2\Delta t}, \dots)}{p(x_{t-\Delta t} = \tilde{x}_{t-\Delta t}, x_{t-2\Delta t} = \tilde{x}_{t-2\Delta t}, \dots)}.$$

- Instead of learning a one-way 'rule' to predict the future, we train the model to understand the overall patterns of short time windows (sequences).

$$\underbrace{\hat{p}_\theta(x_t, x_{t-\Delta t}, x_{t-2\Delta t}, \dots)}_{\text{Deep Generative Model}} \approx \underbrace{p(x_t, x_{t-\Delta t}, x_{t-2\Delta t}, \dots)}_{\text{Adjacent time steps}},$$



Generative Forecasting using **Joint** Probability Models

Uncertainty quantification from joint samples

In chaotic systems, the model will eventually lose track of the truth, and we need some metrics tell us **exactly when and why** that is happening without us needing to know the actual future.

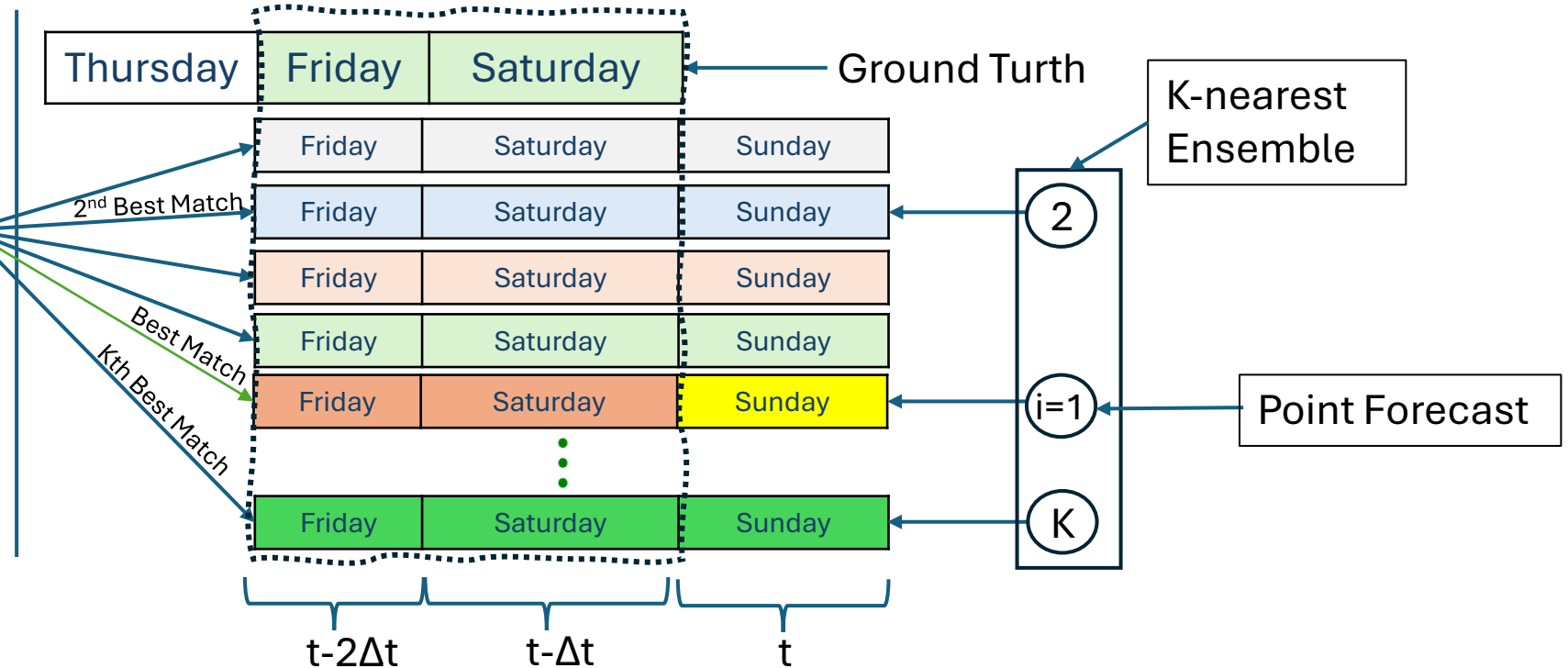
"Top-k" Selection

Generative Model provide us with samples

$$\hat{p}_{\theta}(x_t, x_{t-\Delta t}, x_{t-2\Delta t}, \dots)$$

Sampling

Day 1
Day 2
Day 2



Generative Forecasting using **Joint** Probability Models

1. Ensemble Variance: (On Predicted Forecast)

- First, we need calculate the weighted average of the all the predicted forecasts (sunday) from the k-nearest ensemble.

$$\bar{x}_t = \frac{\sum_{i=1}^k w_i x_t(i)}{\sum_{i=1}^k w_i},$$

wi is the optional importance weights

- Then we can calculate the ensemble variance.
$$\sigma_{\text{ens}}(x_t) = \frac{\sum_{i=1}^k w_i (x_t(i) - \bar{x}_t)^2}{\sum_{i=1}^k w_i},$$

- High Variance Indicates the Model is Uncertain.**

Generative Forecasting using **Joint** Probability Models

2. Autocorrelation : (Between Predicted Forecast and the Previous State)

- Autocorrelation measures how much the state at time t (e.g., Sunday) depends linearly on the state at time $t-\Delta t$

$$AC(x_t, x_{t-\Delta t}) = \frac{\frac{1}{k} \sum_{i=1}^k (x_t^{(i)} - \bar{x}_t) \odot (x_{t-\Delta t}^{(i)} - \bar{x}_{t-\Delta t})}{\sigma_t \odot \sigma_{t-\Delta t}},$$

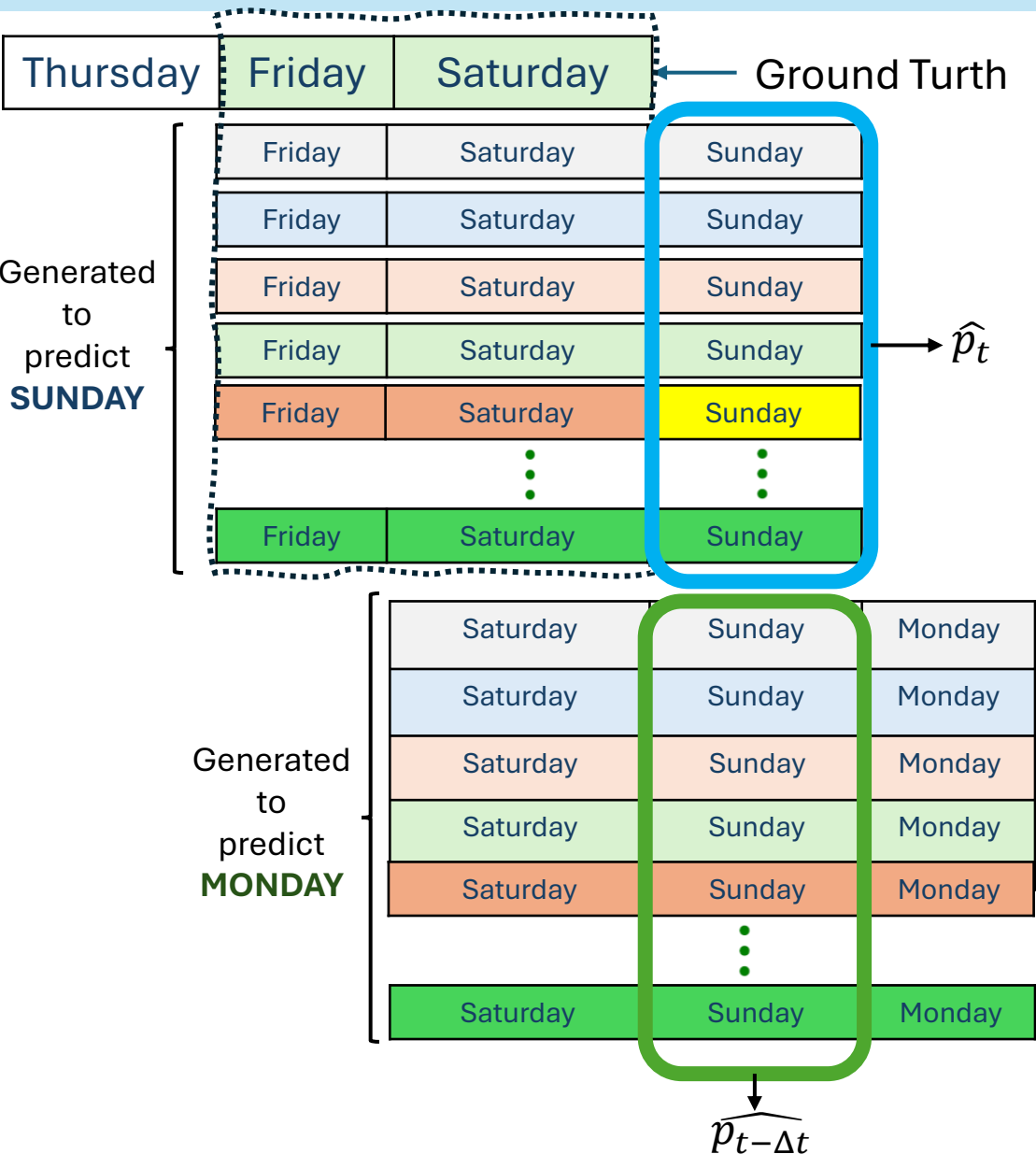
$\bar{x}_t, \bar{x}_{t-\Delta t}$: Empirical Means at t and $t-\Delta t$ respectively

$\sigma_t, \sigma_{t-\Delta t}$: Standard deviations at t and $t-\Delta t$ respectively

If AC is close to 1: This means the system has **strong momentum** and changes slowly relative to time.

If AC is close to 0: This means the system is **highly volatile** or is being dominated by random noise.

Wasserstein Drift



- **Wasserstein Drift** measures the model's self-consistency by tracking how much overlapping predictions for the exact same time step diverge.
- This provides an early warning signal for forecast failure without needing actual ground truth data.

$WD_t = W_2(\hat{p}_t, \hat{p}_{t-\Delta t})$ -- estimated in practice using a Sinkhorn-regularized optimal transport solver.

- A higher score means the overlapping predictions are drifting apart, indicating the model is getting confused.
- The signed version of the Wasserstein distance using changes in ensemble variance:

$$\widetilde{WD}_t = \begin{cases} -WD_t, & \sigma_{\text{ens}}(x_{t+\Delta t}) < \sigma_{\text{ens}}(x_t), \\ WD_t, & \text{otherwise,} \end{cases}$$

- The cumulative Wasserstein :

$$WD_{\text{recon}}(x_t; \{x_s : s \leq t\}) = \sum_{s=1}^t \widetilde{WD}_s.$$

Extension to Joint Generative Framework

1. Conditional Joint Models.

- The existing baseline conditional models use past data to predict **only the single next time step**.
- This extension to the joint generative framework, **conditional joint models** takes past data into the model to generate a full overlapping time window and then selects the generated window that best matches our known recent past.

$$\hat{p}_{\theta}(x_t, x_{t-\Delta t}, \dots, x_{t-(n-1)\Delta t}) \approx p(x_t, x_{t-\Delta t}, \dots, x_{t-(n-1)\Delta t} \mid x_{t-\Delta t}, x_{t-2\Delta t}, \dots, x_{t-(n-1)\Delta t}),$$

- In this case, the framework uses the observed history **twice** in this extension
 1. **Explicitly:** Input as a conditional
 2. **Implicitly:** During sieving - the picking the best forecast

Lorenz-63 model

$$\begin{aligned}\frac{dx}{dt} &= \sigma(y - x), \\ \frac{dy}{dt} &= x(\rho - z) - y, \\ \frac{dz}{dt} &= xy - \beta z,\end{aligned}$$

- x: The rate of convection
- y: The horizontal temperature variation.
- z: The vertical temperature variation
- σ : Prandtl number
- ρ :Rayleigh number
- β : A geometric factor related to the physical dimensions of the fluid layer

Generating the Data for Training

Choice of Parameters

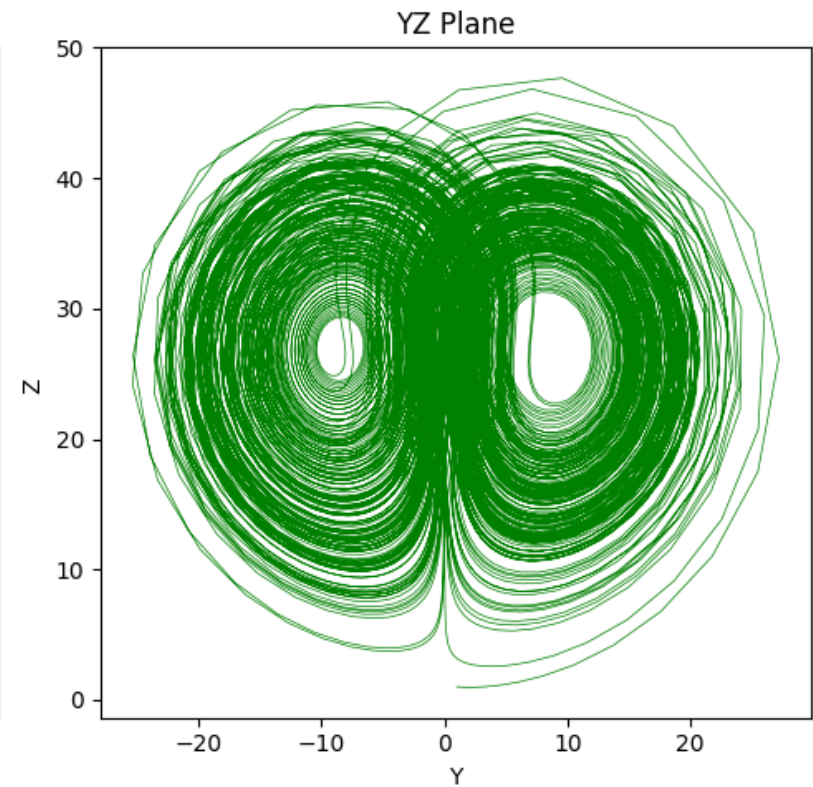
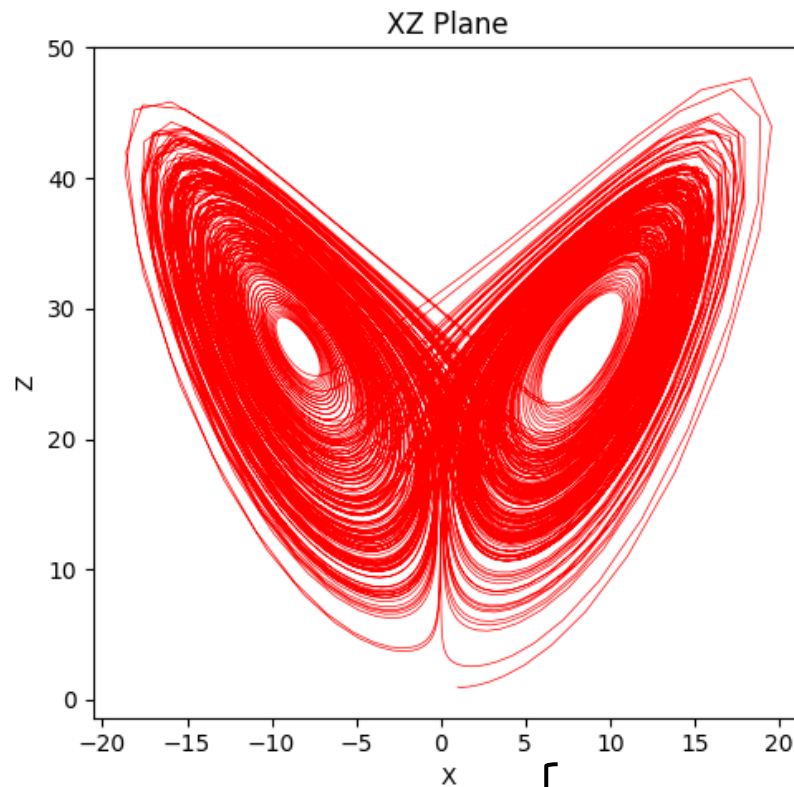
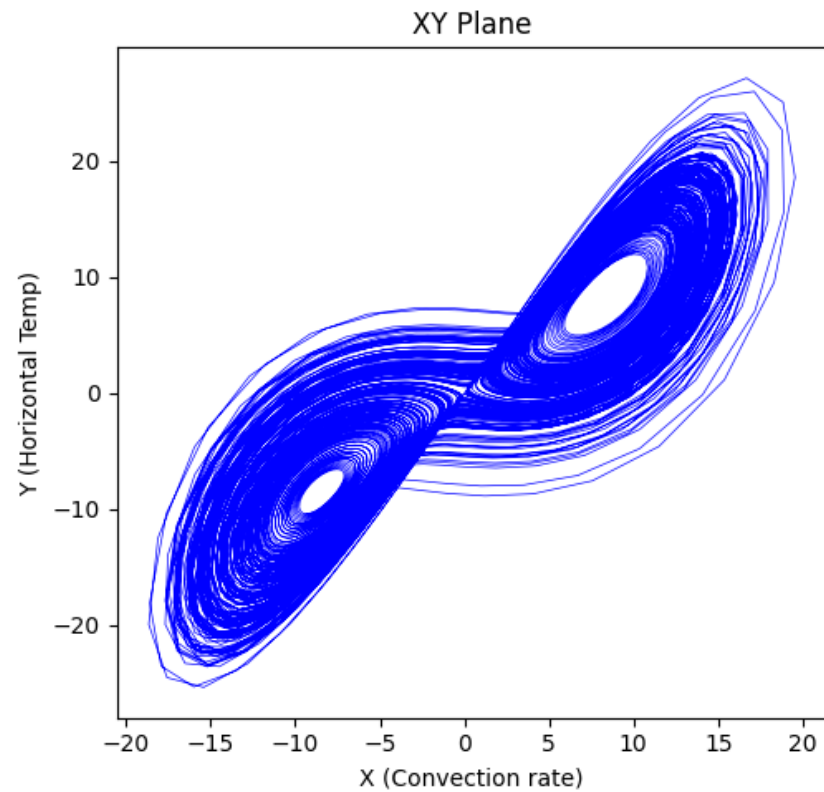
$\sigma= 10$

$\rho = 28$

$\beta = 8/3$

	Research Paper	Current Implementation
Method	RK4	
Time Step Δt	2.5×10^{-2}	
Steps	4×10^6	
Initial Condition	$x,y,z = [1.0, 1.0, 1.0]$	
n-step windows	2	
Joint Windows	999,999 time windows	

Lorenz-63 model



Trajectory

```
array([[1.          , 1.          , 1.          ],  
       [1.07545053, 1.65930813, 0.96861164],  
       [1.2876899 , 2.39988521, 0.96379682],  
       [1.63448705, 3.30797486, 1.00201622],  
       [2.13310847, 4.47105098, 1.11389662]])
```

Joint Windows

```
[[1.          1.          1.          1.07545053 1.65930813 0.96861164]  
 [1.07545053 1.65930813 0.96861164 1.2876899  2.39988521 0.96379682]  
 [1.2876899  2.39988521 0.96379682 1.63448705 3.30797486 1.00201622]  
 [1.63448705 3.30797486 1.00201622 2.13310847 4.47105098 1.11389662]  
 [2.13310847 4.47105098 1.11389662 2.81735656 5.98633224 1.35396999]]
```

NADE Implementation to Forecast Lorenz System

Data

Number of Datapoints (Time Windows) = 10^6

NADE Architecture

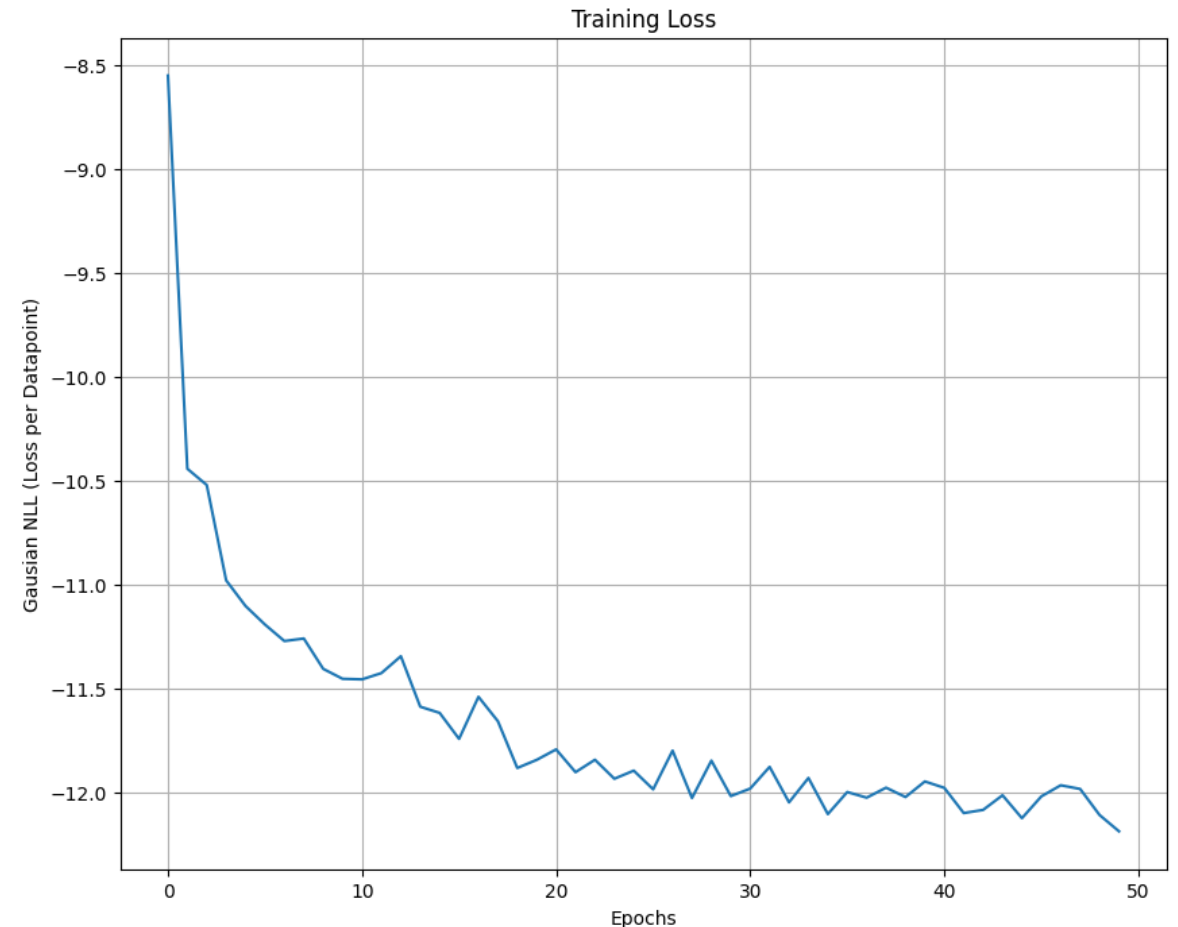
Hidden Layer Dimension = 1000

Training

- Batch Size = 500
- Loss Function: Gaussian Negative Log Likelihood
- Adam Optimizer with Learning Rate = 0.001
- Epochs = 50

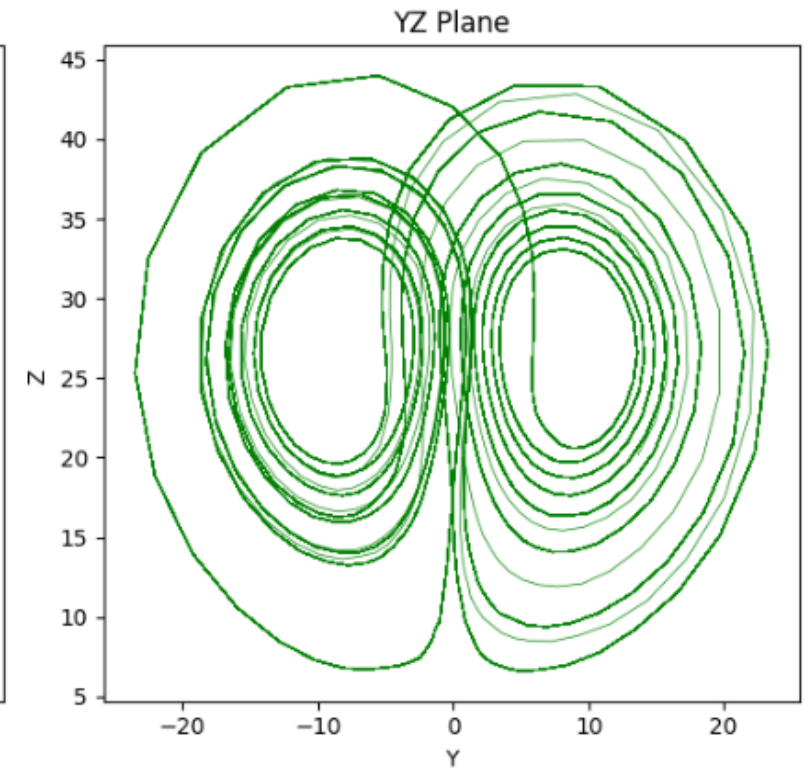
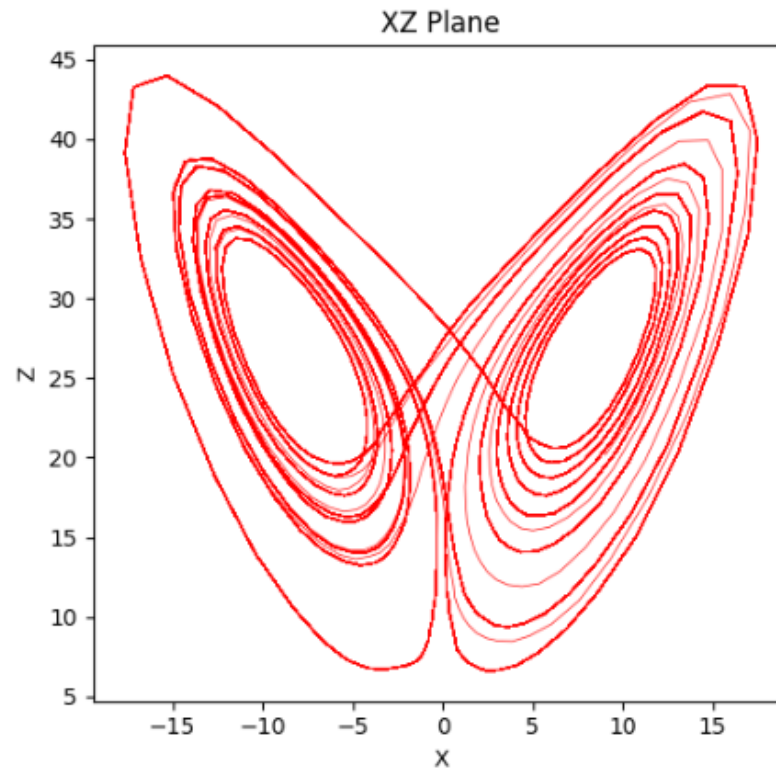
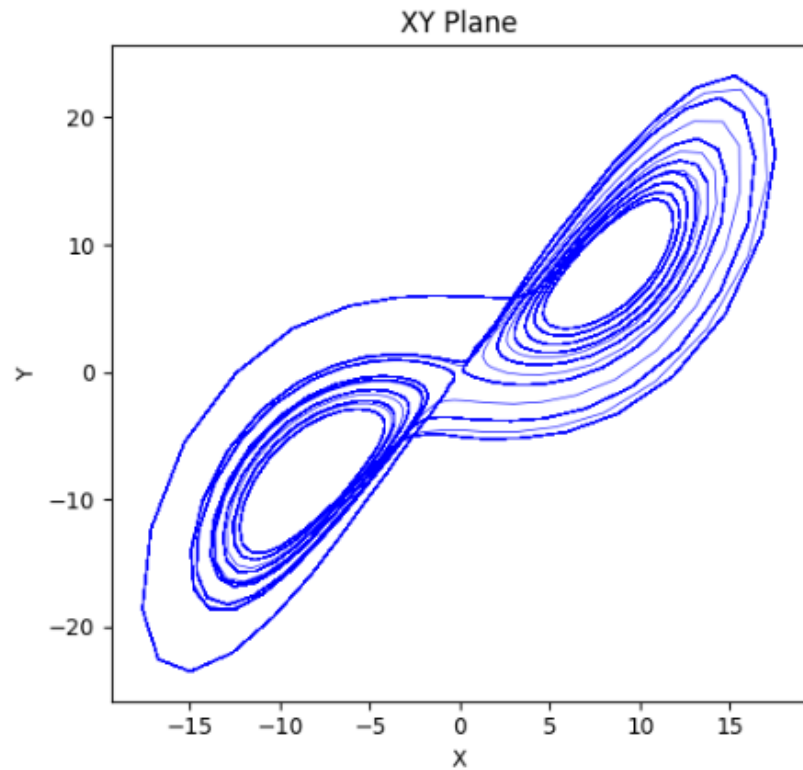
Data Generation

Total Data Points Generated = 10^6
(Only Once)



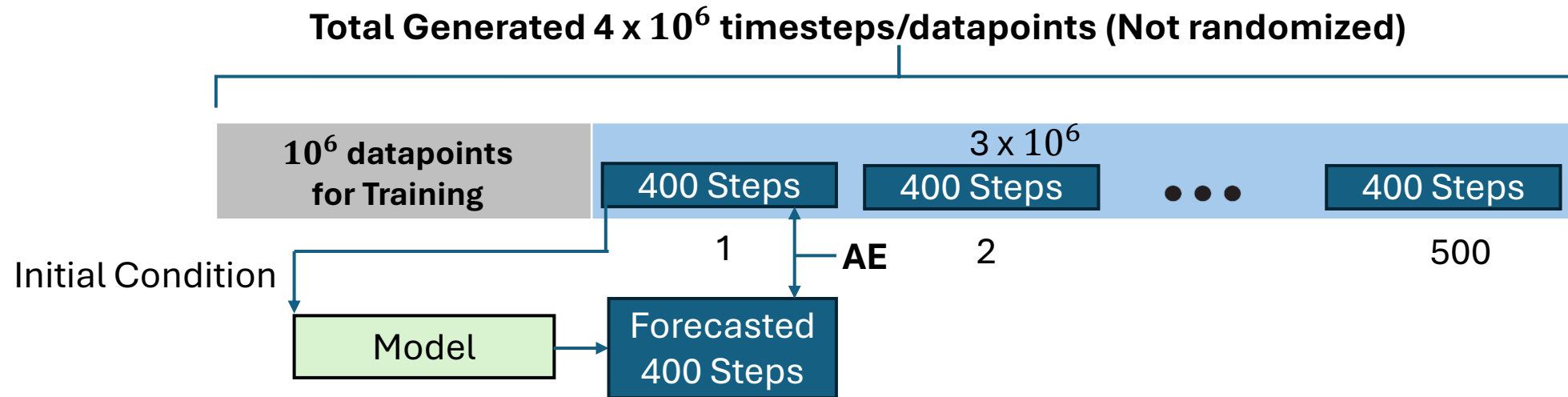
NADE Implementation to Forecast Lorenz System

Trained on (10^6) time steps and forecasted from $(10^6 + 1)$ for 50000 steps



Testing

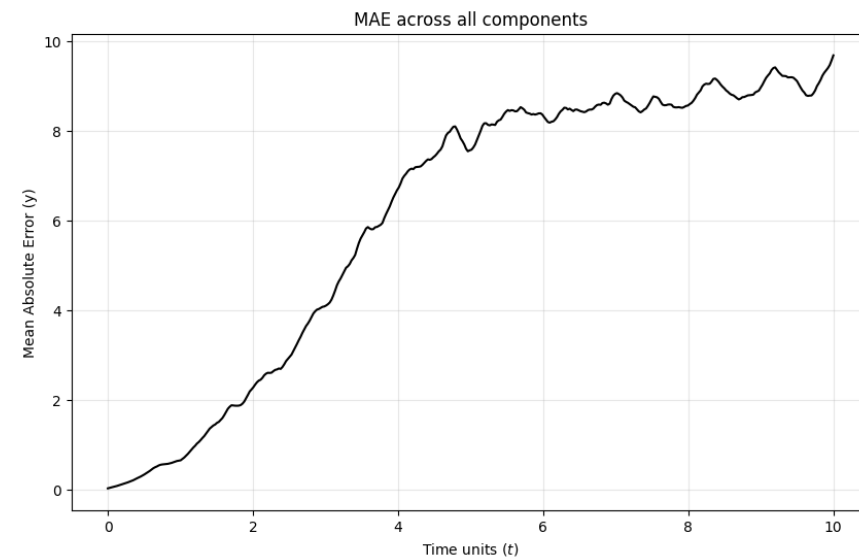
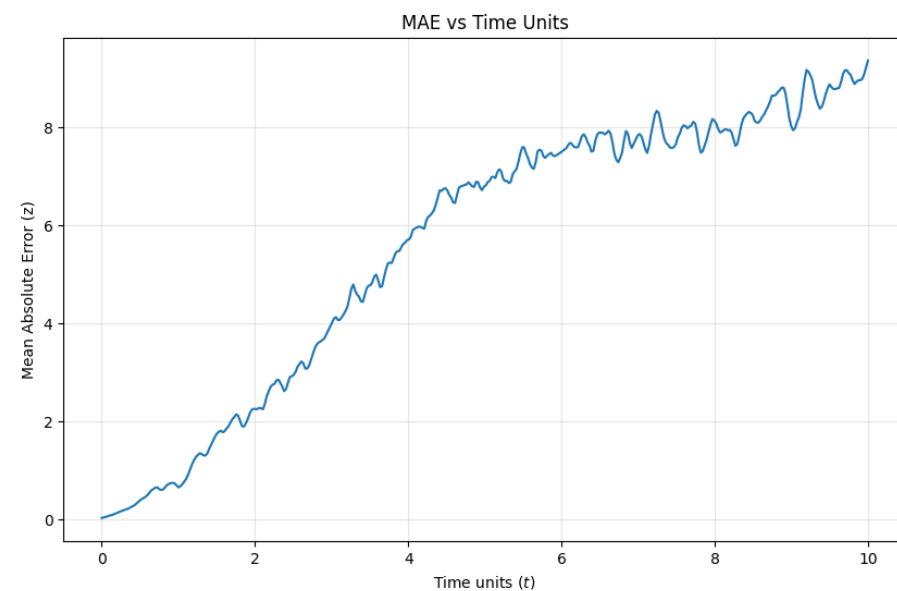
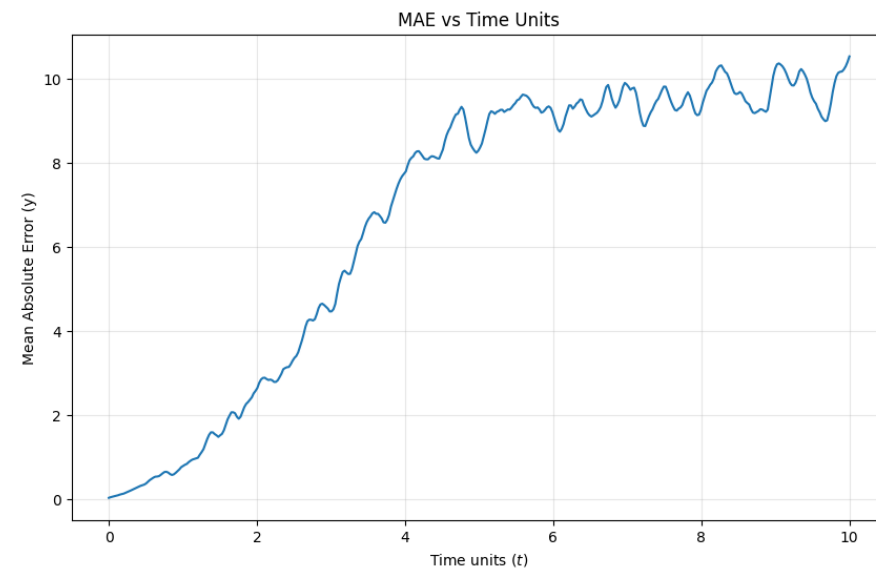
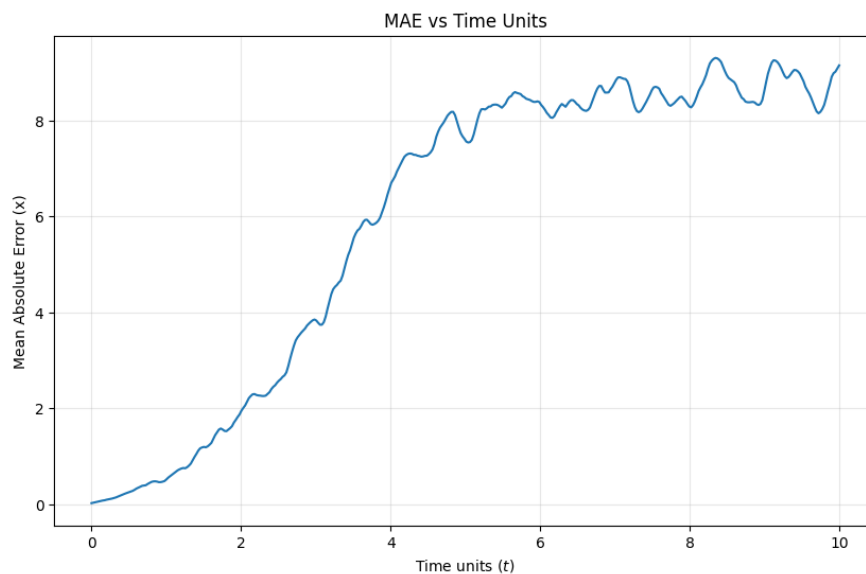
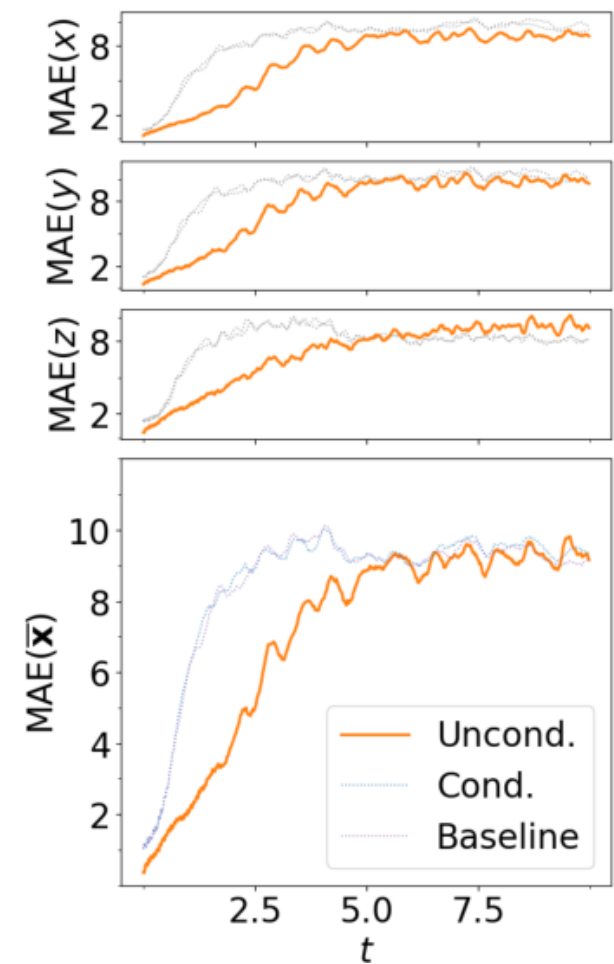
- For testing, we consider 500 unseen initial conditions drawn uniformly forming **nonoverlapping trajectories**.
- Each initial state is forecasted for 400 steps.



- Mean absolute error (MAE) across an **ensemble of 500 test initial conditions** drawn from the attractor but not used for training.

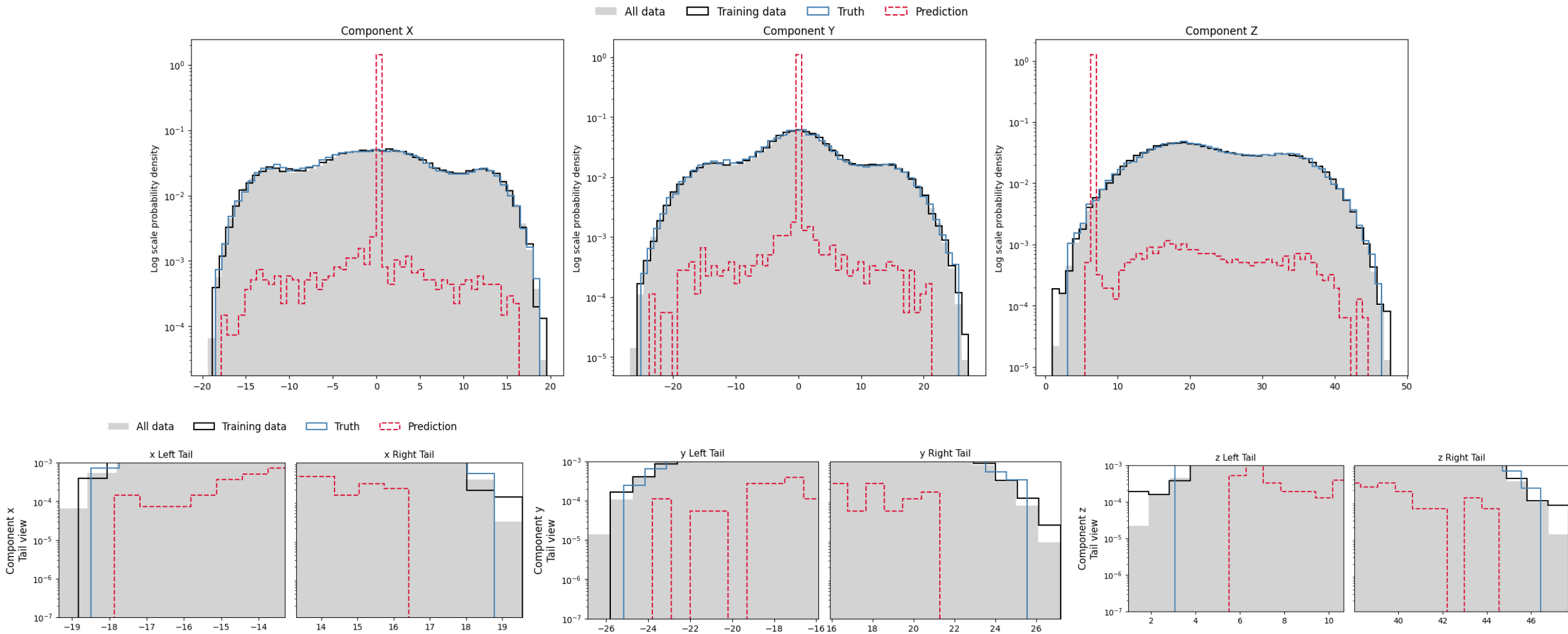
Testing

Research Paper Plots



Distribution adherence test

- NADE was trained with 40000-time windows.
- Forecasted for 20000 steps from 40000 to 60000.



Estimation UQ metrics

- NADE was trained with 1e6 time windows.
- For testing, we consider 500 unseen initial conditions drawn uniformly forming **nonoverlapping trajectories**. Each initial state is forecasted for 400 steps.

Algorithm: Ensemble Variance

Input: $X \in \mathbb{R}^{K \times D}$

Output: $v_{\text{avg}} \in \mathbb{R}$

```
1: for  $j = 1, \dots, D$  do
2:    $\mu_j \leftarrow \frac{1}{K} \sum_{i=1}^K X_{i,j}$ 
3:    $\sigma_j^2 \leftarrow \frac{1}{K-1} \sum_{i=1}^K (X_{i,j} - \mu_j)^2$ 
4: end for
5:  $v_{\text{avg}} \leftarrow \frac{1}{D} \sum_{j=1}^D \sigma_j^2$ 
6: return  $v_{\text{avg}}$ 
```

Algorithm: Ensemble Autocorrelation

Input: $T \in \mathbb{R}^{K \times D}$ (tails), $H \in \mathbb{R}^{K \times D}$ (heads)

Output: $c_{\text{avg}} \in \mathbb{R}$

```
1: for  $j = 1, \dots, D$  do
2:    $\mu_{T,j} \leftarrow \frac{1}{K} \sum_{i=1}^K T_{i,j}$ 
3:    $\mu_{H,j} \leftarrow \frac{1}{K} \sum_{i=1}^K H_{i,j}$ 
4:    $\sigma_{T,j} \leftarrow \sqrt{\frac{1}{K-1} \sum_{i=1}^K (T_{i,j} - \mu_{T,j})^2}$ 
5:    $\sigma_{H,j} \leftarrow \sqrt{\frac{1}{K-1} \sum_{i=1}^K (H_{i,j} - \mu_{H,j})^2}$ 
6:    $\text{num}_j \leftarrow \frac{1}{K} \sum_{i=1}^K (T_{i,j} - \mu_{T,j})(H_{i,j} - \mu_{H,j})$ 
7:    $\text{den}_j \leftarrow \sigma_{T,j} \cdot \sigma_{H,j}$ 
8:    $c_j \leftarrow \frac{\text{num}_j}{\text{den}_j}$ 
9: end for
10:  $c_{\text{avg}} \leftarrow \frac{1}{D} \sum_{j=1}^D c_j$ 
11: return  $c_{\text{avg}}$ 
```

Algorithm: Average Wasserstein Drift

Input: $T \in \mathbb{R}^{K \times D}$ (current tails), $H \in \mathbb{R}^{K \times D}$ (previous heads)

Output: $w_{\text{avg}} \in \mathbb{R}$

```
1: for  $j = 1, \dots, D$  do
2:    $w_j \leftarrow \mathcal{W}_1(T_{:,j}, H_{:,j})$ 
3: end for
4:  $w_{\text{avg}} \leftarrow \frac{1}{D} \sum_{j=1}^D w_j$ 
5: return  $w_{\text{avg}}$ 
```

Algorithm: Signed Drift

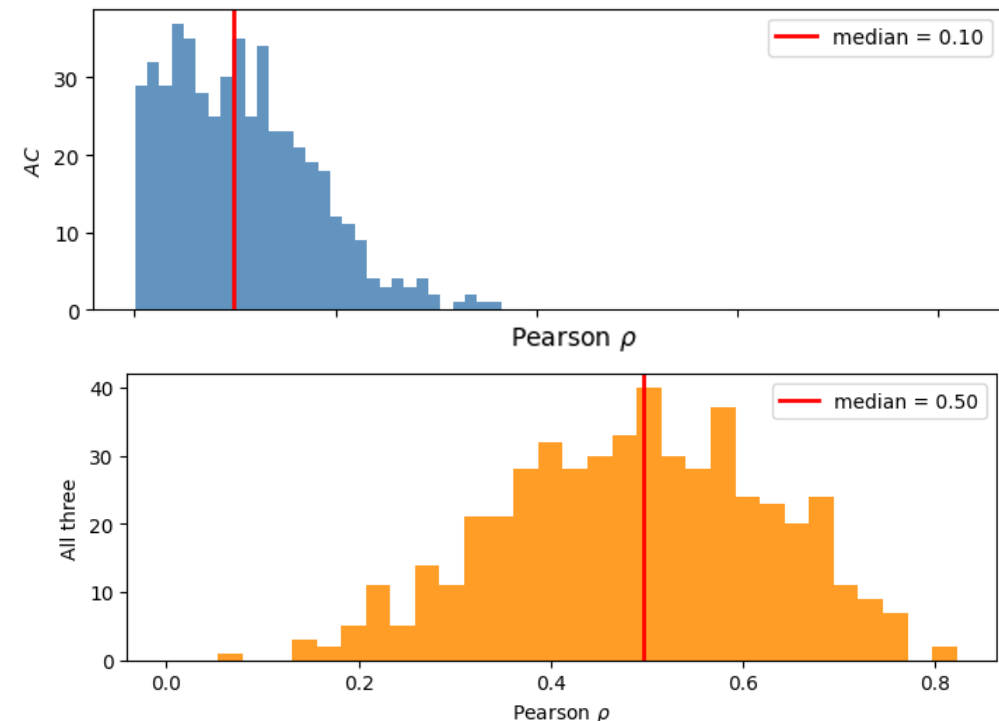
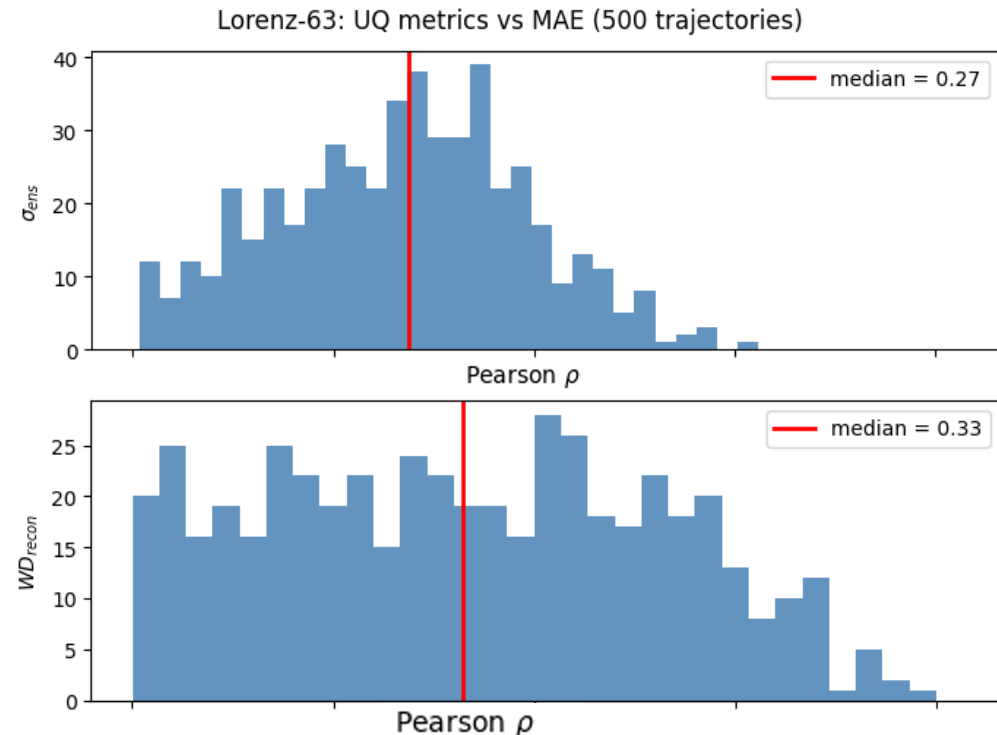
Input: $w \in \mathbb{R}$ (Wasserstein drift), $v_c \in \mathbb{R}$ (current variance), $v_p \in \mathbb{R}$ (previous variance)

Output: $s \in \mathbb{R}$

```
1: if  $v_c < v_p$  then
2:    $s \leftarrow -w$ 
3: else
4:    $s \leftarrow w$ 
5: end if
6: return  $s$ 
```

Estimation UQ metrics

- Ensemble Variance, Autocorrelation and Wasserstein Drift has been calculated for 500X400 grid.
- From Ground Truth MAE is also calculated for this 500X400 grid.
- Performed linear regression on the 500 X 400 grid to map UQ metrics directly to the Mean Absolute Error (MAE).
- Applied linear regression to model **MAE (response variable)** using the three UQ metrics (regressors).
- Configured four regression models per trajectory: one for each individual metric, and one multiple regression combining all three.
- Extracted the Pearson correlation coefficient per trajectory.



Implementation of Conditional Joint Model

Data

Number of Datapoints (Time Windows) = 10^6

NADE Architecture

Hidden Layer Dimension = 1000

Training

- Batch Size = 500
- Loss Function: Gaussian Negative Log Likelihood
- Adam Optimizer with Learning Rate = 0.001
- Epochs = 50

Data Generation

Total Data Points Generated = 1000 per forecast step

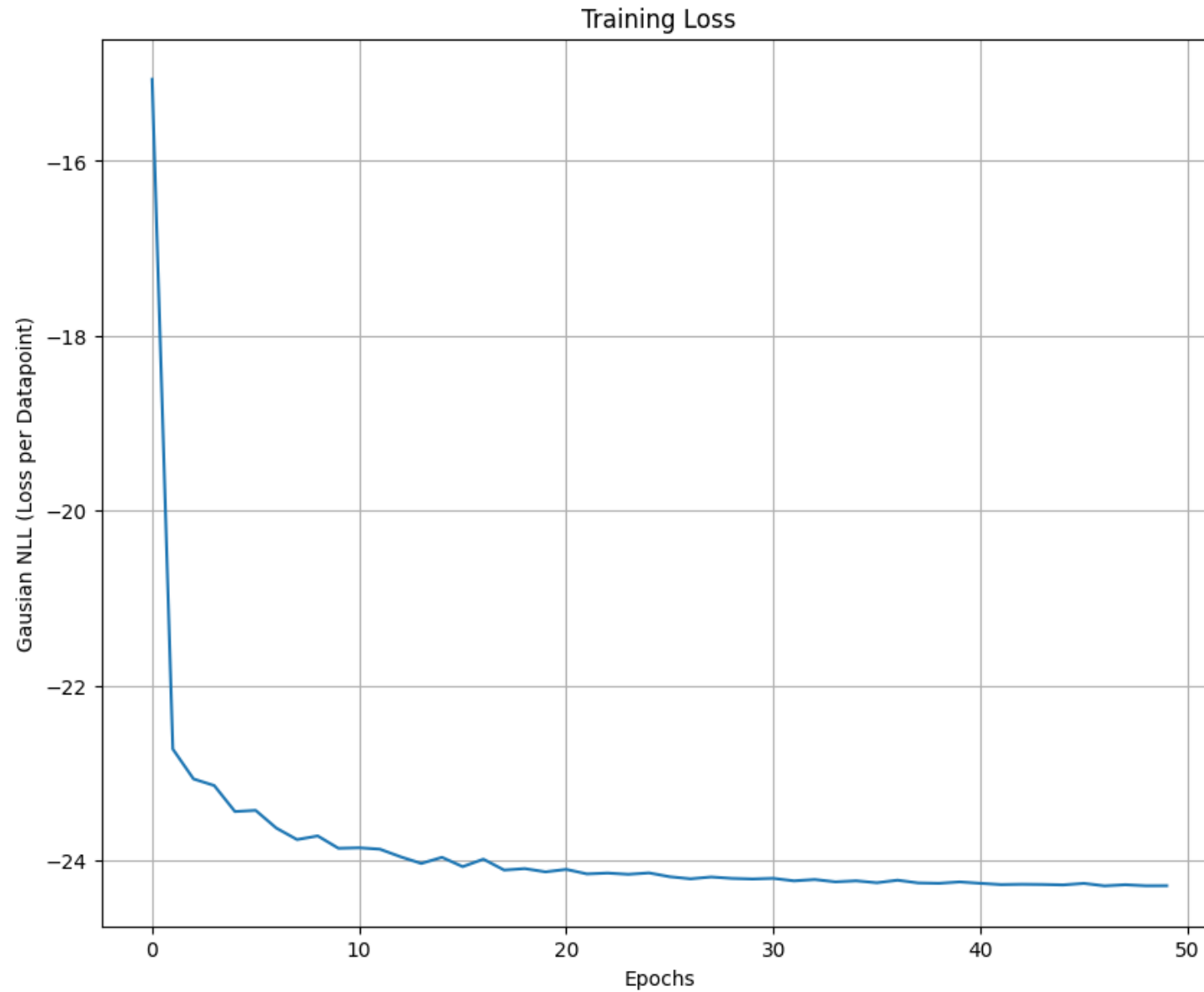
Algorithm: Forecasting via Conditional Joint Marginalization

Input: Trained conditional generative model $\mathcal{J}_\theta(\cdot|\text{history})$, observed n -lag sequence $\tilde{x}_{t-\Delta t}^{\{n-1:0\}}$

Initialization: None required outside the loop for the conditional model.

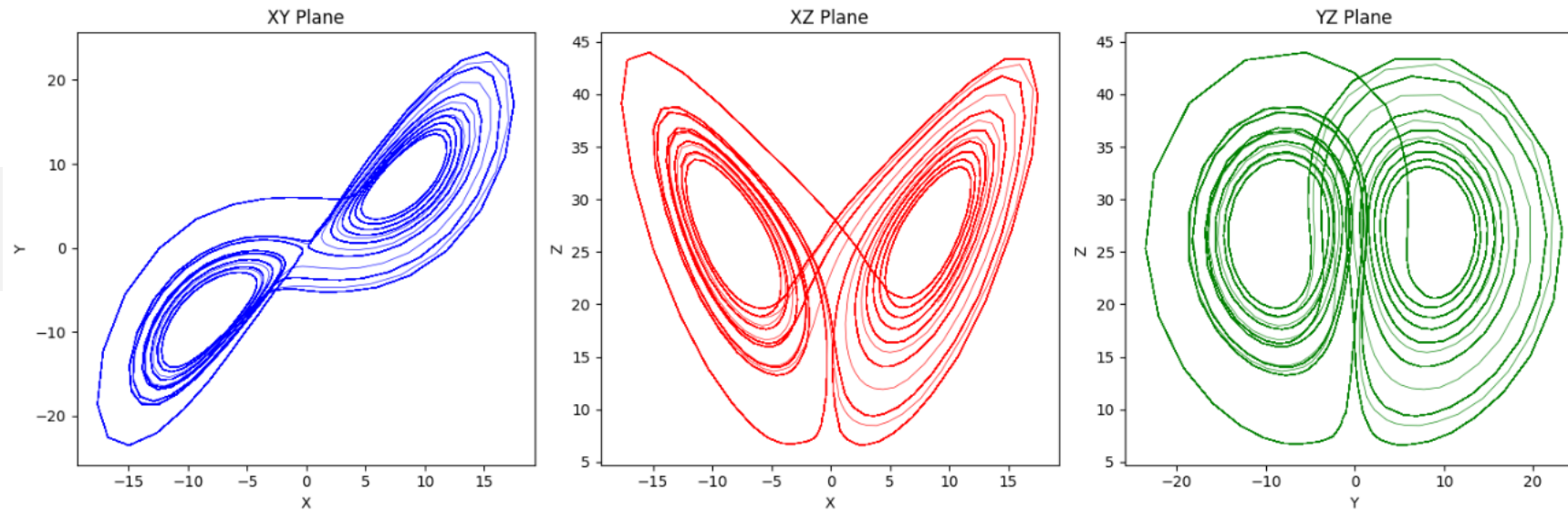
1. **repeat**
2. ▷ *Conditioning & Sampling:* Generate a batch of N joint samples conditioned directly on the current observed history:
$$\{\hat{x}_t^{\{n-1:0\}}(j)\}_{j=1}^N \sim \mathcal{J}_\theta(\cdot|\tilde{x}_{t-\Delta t}^{\{n-1:0\}})$$
3. ▷ *Match on trailing $n - 1$ states:* Find the sample whose "tail" best matches the observation:
$$j^* \leftarrow \arg \min_j \left\| \hat{x}_t^{\{n-1:1\}}(j) - \tilde{x}_{t-\Delta t}^{\{n-2:0\}} \right\|$$
4. ▷ *Retain head state:* Extract the predicted next state:
$$x_t^* \leftarrow \hat{x}_t^{\{0\}}(j^*)$$
5. ▷ *Shift window and append forecast:* $\tilde{x}_t^{\{n-1:0\}} \leftarrow \tilde{x}_{t-\Delta t}^{\{n-1:1\}} \oplus x_t^*$
6. $t \leftarrow t + \Delta t$
7. **until** desired forecast horizon reached
8. **Return:** retained trajectory $\{x_t^*\}$

Implementation of Conditional Joint Model

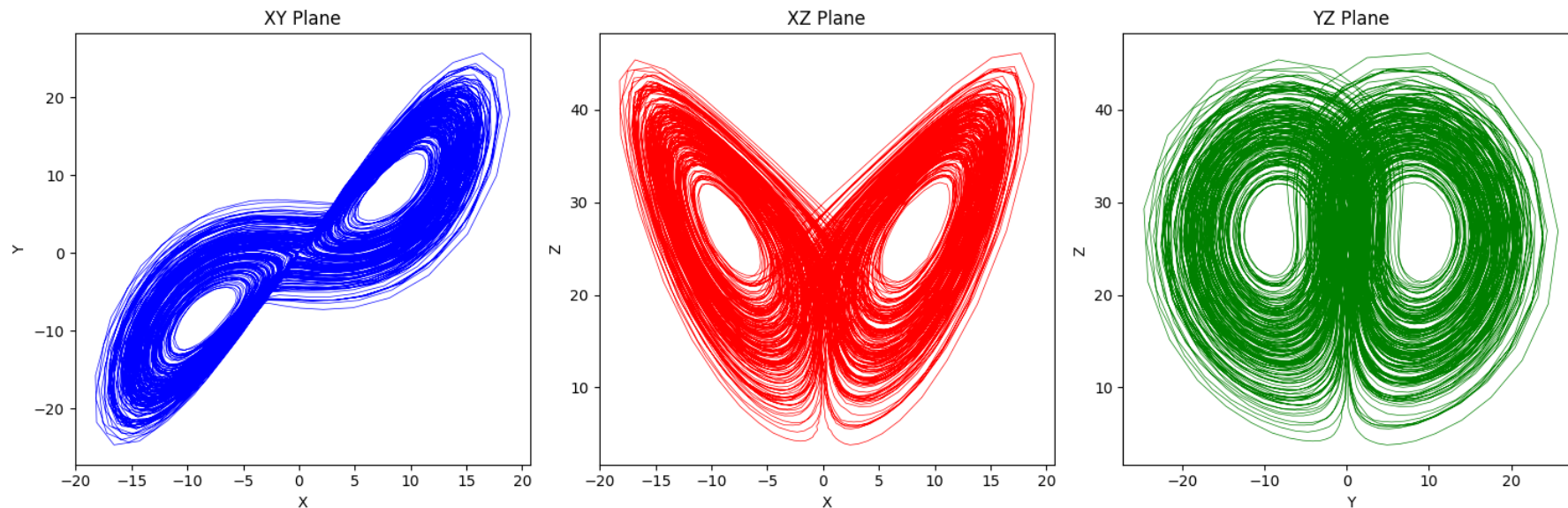


Implementation of Conditional Joint Model

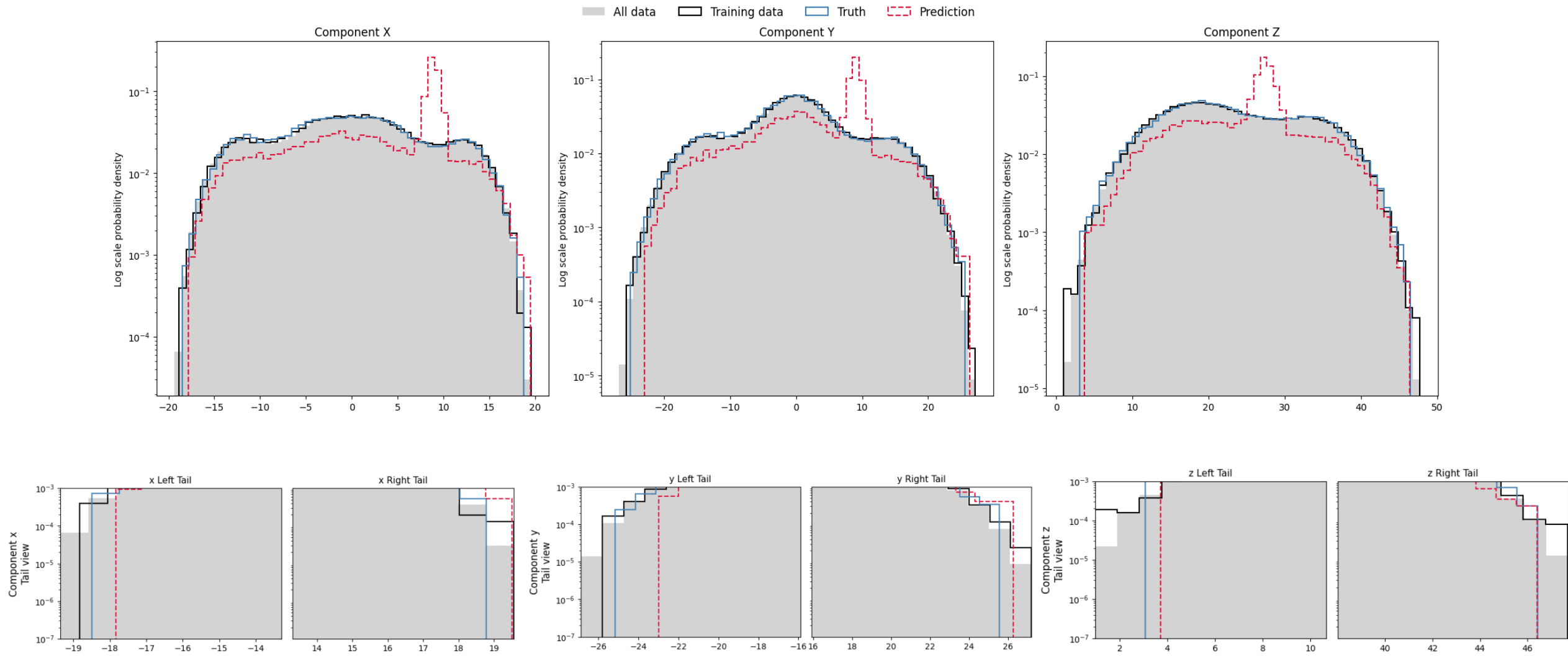
**Unconditional Joint
Generative Model**



**Conditional Joint
Generative Model**



Distribution adherence test



Future Work

- Increase the complexity of the NADE architecture and evaluate its performance.
- Implement the model for the KS equation.
- Extend the implementation to high-dimensional datasets such as ERA5.

Thank You